

Topics: [Core](#) · [AOP](#) · [JDBC](#) · [MVC](#) · [Bo](#)

This guide covers every topic from the SDET v3 Curriculum — Spring Core, Boot and MVC sheet — with full theory explanations, key definitions, and working Java code examples for each subtopic.

Table of Contents

Topic 1: Dependency Injection

- 1.1 What is Dependency Injection?
- 1.2 Setter Injection
- 1.3 Constructor Injection
- 1.4 Autowiring
- 1.5 Injecting Collections
- 1.6 Inner Beans
- 1.7 Inheritance, Interfaces & Bean Scope

Topic 2: Spring AOP

- 2.1 What is AOP & Why Use It?
- 2.2 AOP Key Terminology
- 2.3 Types of Advice
- 2.4 Pointcut Expressions
- 2.5 Complete AOP Example

Topic 3: Reading Values from Property Files

- 3.1 Why Externalise Configuration?
- 3.2 PropertyPlaceholderConfigurer
- 3.3 @Value & @PropertySource
- 3.4 ResourceBundleMessageSource (i18n)

Topic 4: Spring JDBC and Transactions

- 4.1 Why Spring JDBC?
- 4.2 JdbcTemplate — Executing Queries
- 4.3 NamedParameterJdbcTemplate
- 4.4 Transaction Management
- 4.5 Propagation & Isolation

Topic 5: Spring MVC Fundamentals

- 5.1 Web Application Architecture
- 5.2 Application Servers & Tomcat
- 5.3 Servlets
- 5.4 JSP (JavaServer Pages)
- 5.5 JSTL (JSP Standard Tag Library)

Topic 6: Spring MVC with Spring Boot

- 6.1 Initialising & Auto-Configuring Spring Boot
- 6.2 Configuring Web App using Spring Boot
- 6.3 Spring MVC Tag Libraries
- 6.4 View Resolvers
- 6.5 Controllers
- 6.6 Internationalization
- 6.7 Data Binding, Validation & Exception Handling

Topic 7: Spring Data JPA

- 7.1 What is JPA & Spring Data JPA?
- 7.2 Creating Spring Data JPA Projects
- 7.3 Entity Class & Entity Mapping

- 7.4 Repository (CrudRepository, JpaRepository)
- 7.5 JPQL
- 7.6 Criteria Query
- 7.7 Spring Data JPA & Transactions
- 7.8 Testing

Topic 8: Spring REST

- 8.1 What is REST?
- 8.2 REST Basics & Annotations
- 8.3 Handling Request and Response
- 8.4 Global Error Handling
- 8.5 Spring REST HTTP Methods
- 8.6 Testing Spring REST Methods

Topic 1: Dependency Injection

1.1 What is Dependency Injection?

Dependency Injection (DI) is a design pattern and one of the core principles of the Spring Framework. In traditional programming, a class creates its own dependencies using the **new** keyword. This tightly couples the class to its dependencies, making it difficult to test and maintain.

With DI, a class declares what it needs (its dependencies), and an external entity — the **Spring IoC (Inversion of Control) Container** — provides those dependencies at runtime. The control of creating objects is *inverted* from the class itself to the container.

Concept	Without DI	With DI (Spring)
Object creation	Class creates its own objects (new)	Container creates & injects
Coupling	Tightly coupled	Loosely coupled
Testability	Hard to mock dependencies	Easy to mock & unit test
Configuration	Hardcoded	Externalised (XML / annotations)

Benefits of DI: Loose coupling between classes, easier unit testing by injecting mock objects, better code reusability, and centralised configuration of the application.

***Note:** The Spring IoC container reads the bean definitions (XML or Java annotations) and manages the full lifecycle of beans: instantiation, dependency injection, and destruction.*

1.2 Setter Injection

In **Setter Injection**, the Spring container first creates the bean using a **no-argument constructor**, and then calls the public setter methods to inject the required dependencies. This is useful for *optional* dependencies.

How it works:

- Spring creates the bean instance using the default (no-arg) constructor.
- Spring reads the <property> tags (XML) or @Autowired on setter (annotation).
- Spring calls the setter method and passes the dependency value.

***Tip:** Use setter injection for optional dependencies. It also allows re-injection after object creation.*

```
// ■■■ Model classes ■■■  
public class Engine {  
    private String type;  
    public Engine() { this.type = "V8"; }  
    public void start() { System.out.println(type + " Engine started!"); }  
    public void setType(String type) { this.type = type; }  
}  
  
public class Car {  
    private Engine engine; // dependency  
  
    // No-arg constructor (required for setter injection)  
    public Car() {}  
  
    // Setter - Spring will call this  
    public void setEngine(Engine engine) {  
        this.engine = engine;  
    }  
  
    public void drive() {  
        engine.start();  
        System.out.println("Car is driving!");  
    }  
}  
  
// ■■ XML Configuration (applicationContext.xml) ■■■  
/*  
<beans>  
    <bean id="engine" class="com.example.Engine">  
        <property name="type" value="V6"/>  
    </bean>  
  
    <bean id="car" class="com.example.Car">  
        <!-- setter injection: calls car.setEngine(engine bean) -->  
        <property name="engine" ref="engine"/>  
    </bean>  
</beans>  
*/  
  
// ■■ Equivalent Java Config (@Configuration) ■■■  
@Configuration  
public class AppConfig {  
    @Bean  
    public Engine engine() {  
        Engine e = new Engine();  
        e.setType("V6");  
        return e;  
    }  
}
```


Autowiring modes:


```
        private MessageSender sender;
    }

// ■■ @Primary – mark a default bean ████████████████████████████████████████████
@Component
@Primary                               // used when no @Qualifier is specified
public class EmailSender implements MessageSender { ... }
```

Note: `@Autowired(required = false)` tells Spring not to throw an error if no matching bean is found — the field will remain null. Use carefully.

1.5 Injecting Collections

Spring can inject **List**, **Set**, **Map**, and **Properties** types directly into beans. This is useful when a bean needs a group of values or references. Spring also supports injecting *all beans of a given type* automatically into a List.

prototype	New instance every getBean	Stateful objects, user sessions
request	One per HTTP request	Web apps — request-scoped data
session	One per HTTP session	Web apps — user session data
application	One per ServletContext	Web apps — app-wide shared data

Topic 2: Spring AOP (Aspect-Oriented Programming)

2.1 What is AOP and Why Use It?

Aspect-Oriented Programming (AOP) is a programming paradigm that allows developers to separate **cross-cutting concerns** from the core business logic. Cross-cutting concerns are features that affect multiple parts of an application but do not belong to any single module — for example: **logging**, **security**, **transaction management**, **caching**, and **performance monitoring**.

Without AOP, you would need to copy-paste the same logging or security code into every method. AOP lets you define this behaviour *once* in an **Aspect** and apply it declaratively to any method matching a pattern.

Problem AOP Solves:

```
// WITHOUT AOP – logging scattered everywhere (bad practice)
public class OrderService {
    public void placeOrder(Order o) {
        System.out.println("LOG: placeOrder called");    // duplicated
        // ... business logic ...
        System.out.println("LOG: placeOrder finished"); // duplicated
    }
    public void cancelOrder(int id) {
        System.out.println("LOG: cancelOrder called"); // duplicated
        // ... business logic ...
    }
}

// WITH AOP – logging in ONE place, applied to ALL methods
@Aspect @Component
public class LoggingAspect {
    @Before("execution(* com.example.service.*(..))")
    public void log(JoinPoint jp) {
        System.out.println("LOG: " + jp.getSignature().getName() + " called");
    }
}

// OrderService now has zero logging code – clean and focused
```

2.2 AOP Key Terminology

Term	Definition
Aspect	A module that encapsulates a cross-cutting concern (e.g., LoggingAspect class)
Join Point	A specific point during program execution where an aspect can be applied (e.g., method execution)
Advice	The action taken by an aspect at a join point (Before, After, Around, etc.)
Pointcut	An expression that selects which join points the advice should apply to
Weaving	The process of linking aspects with application code (Spring does this at runtime via proxies)
Target Object	The object whose method is being intercepted by the aspect
Proxy	Spring creates a proxy object wrapping the target; AOP advice runs via this proxy

2.3 Types of Advice

Spring AOP supports five types of advice, each running at a different point relative to the matched method execution:

Advice Type	Annotation	When It Runs
Before	@Before	Before the method executes
After	@After	After the method (whether success or exception)
After Returning	@AfterReturning	Only if method returns successfully
After Throwing	@AfterThrowing	Only if method throws an exception
Around	@Around	Before AND after; most powerful — can change return value

2.4 Pointcut Expressions

Pointcut expressions use the **execution()** designator to match methods. The pattern is: **execution([modifiers] returnType [class].method(params))**

```
// Match any method in any class in the service package
execution(* com.example.service.*(..))

// Match only public methods returning void in UserService
execution(public void com.example.service.UserService.*(..))

// Match methods that start with "get" anywhere
execution(* get*(..))

// Match methods with exactly one String parameter
execution(* *(String))

// Combine pointcuts with && || !
@Pointcut("execution(* com.example.service.*(..))")
public void serviceLayer() {}

@Pointcut("execution(* com.example.repository.*(..))")
public void repositoryLayer() {}

@Before("serviceLayer() || repositoryLayer()")
public void logAll(JoinPoint jp) { ... }
```

2.5 Complete AOP Example

Topic 3: Reading Values from Property Files

3.1 Why Externalise Configuration?

Hard-coding values like database URLs, passwords, or API keys inside Java classes is a bad practice. If you need to change them (e.g., moving from dev to production), you must recompile the code. **Externalising configuration** into **.properties** files allows you to change values without touching source code.

- Separate configuration from code (12-Factor App principle).
- Different property files per environment: dev, test, production.
- Sensitive values (passwords) can be kept outside version control.

3.2 PropertyPlaceholderConfigurer (XML approach)

In classic Spring XML configuration, **PropertyPlaceholderConfigurer** (or its successor **PropertySourcesPlaceholderConfigurer**) reads a .properties file and makes its key-value pairs available as **#{key}** placeholders in XML bean definitions.

3.3 @Value and @PropertySource (Annotation approach)

@PropertySource loads a .properties file into the Spring Environment. **@Value("\${key}")** injects the value of a specific key into a field or constructor parameter. This is the modern, annotation-driven way.

Topic 4: Spring JDBC and Transactions

4.1 Why Spring JDBC?

Traditional JDBC requires a lot of repetitive boilerplate: loading the driver, opening connections, creating statements, handling `ResultSet`s, and closing resources in finally blocks. If you forget to close a connection, you get a resource leak.

Spring's `JdbcTemplate` eliminates all this boilerplate. It handles connection management, exception translation (converting checked `SQLException`s into unchecked Spring `DataAccessException`s), and resource cleanup automatically. You only write the SQL and the result mapping.

4.2 `JdbcTemplate` — Executing Queries


```

@Repository
public class EmployeeDaoNamed {

    @Autowired
    private NamedParameterJdbcTemplate namedJdbc;

    public List<Employee> findBySalaryRange(double min, double max) {
        String sql = "SELECT * FROM employee " +
            "WHERE salary BETWEEN :minSal AND :maxSal";

        MapSqlParameterSource params = new MapSqlParameterSource()
            .addValue("minSal", min)
            .addValue("maxSal", max);

        return namedJdbc.query(sql, params, new EmployeeRowMapper());
    }

    public int saveWithNamed(Employee e) {
        String sql = "INSERT INTO employee(name, department, salary) " +
            "VALUES(:name, :dept, :salary)";
        SqlParameterSource params = new BeanPropertySqlParameterSource(e);
        // BeanPropertySqlParameterSource maps bean property names to :params
        return namedJdbc.update(sql, params);
    }
}

```

4.4 Transaction Management

A **transaction** is a sequence of operations that must either all succeed or all fail together — this is the **ACID** principle (Atomicity, Consistency, Isolation, Durability). Spring's **@Transactional** annotation makes it easy to manage transactions declaratively without writing any transaction management code.

When a method annotated with **@Transactional** throws a **RuntimeException** (unchecked), Spring automatically **rolls back** the transaction. For checked exceptions, rollback must be specified explicitly.

Isolation Level	Prevents
READ_UNCOMMITTED	Nothing — can read dirty, non-repeatable, phantom data
READ_COMMITTED	Dirty reads (default in most DBs like MySQL, Oracle)
REPEATABLE_READ	Dirty reads + non-repeatable reads
SERIALIZABLE	All anomalies — safest but slowest

```

@Transactional(
    propagation = Propagation.REQUIRES_NEW,    // always new transaction
    isolation    = Isolation.READ_COMMITTED,    // prevent dirty reads
    timeout      = 30,                          // rollback if takes > 30s
    readOnly     = false
)
public void auditLog(String action) {
    // This runs in its own transaction, independent of the caller's transaction
    // So even if the caller rolls back, this audit log is committed
    auditDao.save(new AuditEntry(action, LocalDateTime.now()));
}

```

Topic 5: Spring MVC Fundamentals

5.1 Web Application Architecture

A typical Java web application follows a layered architecture: the **Presentation Layer** (JSP/HTML), the **Controller Layer** (Servlets / Spring MVC Controllers), the **Service Layer** (business logic), and the **Data Layer** (JDBC / JPA).

Spring MVC implements the **Front Controller** pattern. Every HTTP request goes through a single central servlet — the **DispatcherServlet** — which then delegates to the appropriate controller based on the URL mapping.

Spring MVC Request Flow:

- Browser sends HTTP request → DispatcherServlet receives it
- DispatcherServlet consults HandlerMapping to find the right Controller
- Controller processes the request, calls Service layer, returns ModelAndView
- DispatcherServlet consults ViewResolver to find the View (JSP/Thymeleaf)
- View renders HTML using Model data and sends HTTP response to browser

5.2 Application Servers and Tomcat

An **Application Server** is software that hosts and runs Java web applications. It provides services like HTTP handling, thread management, connection pooling, and Servlet/JSP support.

Apache Tomcat is the most popular **Servlet Container** (also called a web container). It implements the Java Servlet and JSP specifications. Spring Boot embeds Tomcat by default, so you don't need to install a separate server.

Server	Type	Used For
Apache Tomcat	Servlet Container	Spring Boot, Spring MVC apps
JBoss / WildFly	Full Java EE App Server	Enterprise apps with EJBs
GlassFish	Full Java EE App Server	Reference implementation
Jetty	Lightweight Servlet Container	Embedded use, microservices

5.3 Servlets

A **Servlet** is a Java class that handles HTTP requests and generates HTTP responses. It runs inside a Servlet container (Tomcat). Servlets are the foundation of Java web development. Spring MVC's DispatcherServlet is itself a Servlet.

Servlet Lifecycle: `init()` → `service()` [`doGet/doPost`] → `destroy()`. The container calls `init()` once on startup, `service()` for every request, and `destroy()` on shutdown.

Topic 6: Spring MVC with Spring Boot

6.1 Initialising and Auto-Configuring Spring Boot App

Spring Boot is built on top of Spring Framework and drastically reduces the setup and configuration required. Its key features are:

- **Auto-configuration:** Spring Boot detects jars on the classpath and automatically configures beans. E.g., if spring-data-jpa is on the classpath and a DataSource is configured, it auto-creates EntityManager, TransactionManager, etc.
- **Embedded Server:** Tomcat (or Jetty/Undertow) is embedded inside the JAR — no need to deploy a WAR file.
- **Starter POMs:** Curated dependency bundles (e.g., spring-boot-starter-web includes Spring MVC, Tomcat, Jackson).
- **Actuator:** Production-ready features like health checks, metrics endpoints.
- **@SpringBootApplication:** Combines @Configuration + @ComponentScan + @EnableAutoConfiguration.

A **View Resolver** is a Spring MVC component that translates a logical view name (a String returned by the controller, e.g., **"students/list"**) into an actual view resource (e.g., **/templates/students/list.html**). Spring Boot auto-configures the right resolver based on the template engine on the classpath.

View Resolver	Used With	Auto-configured By
ThymeleafViewResolver	Thymeleaf	spring-boot-starter-thymeleaf
InternalResourceViewResolver	JSP	Manual config (Boot has limited JSP support)
FreeMarkerViewResolver	FreeMarker	spring-boot-starter-freemarker
ContentNegotiatingViewResolver	Multiple	Returns JSON or HTML based on Accept header

[illegible]

6.5 Controllers

@Controller is a Spring component that handles HTTP requests. Methods in a controller are mapped to URLs using **@GetMapping**, **@PostMapping**, etc. (shortcuts for **@RequestMapping**). The controller puts data into the **Model** and returns a view name.

[illegible]


```
// WebMvcConfig.java
@Configuration
public class WebMvcConfig implements WebMvcConfigurer {

    // Stores locale in HTTP session
    @Bean
    public LocaleResolver localeResolver() {
        SessionLocaleResolver slr = new SessionLocaleResolver();
        slr.setDefaultLocale(Locale.ENGLISH);
        return slr;
    }

    // Changes locale when ?lang=ta is in the URL
    @Bean
    public LocaleChangeInterceptor localeChangeInterceptor() {
        LocaleChangeInterceptor lci = new LocaleChangeInterceptor();
        lci.setParamName("lang");
        return lci;
    }

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(localeChangeInterceptor());
    }

    @Bean
    public MessageSource messageSource() {
        ResourceBundleMessageSource ms = new ResourceBundleMessageSource();
        ms.setBasename("i18n/messages");
        ms.setDefaultEncoding("UTF-8");
        return ms;
    }
}

// resources/i18n/messages.properties (English)
// app.title=Student Management System
// nav.home=Home
// nav.students=Students
// form.name=Full Name
// form.email=Email Address

// resources/i18n/messages_ta.properties (Tamil)
// app.title=
// nav.home=
// nav.students=

// Thymeleaf template – use #{key} for i18n messages
// <title th:text=#{app.title}>Student System</title>

// <a th:href="@{/(lang=en)}">English</a>
// <a th:href="@{/(lang=ta)}"></a>
```

Data Binding maps HTTP request parameters (form fields) directly to a Java object's properties. **Validation** checks that the data meets business rules using Bean Validation annotations. **@ControllerAdvice** provides a central place to handle exceptions across all controllers.

[illegible]

```
@ControllerAdvice    // applies to ALL controllers
public class GlobalExceptionHandler {

    // Handle custom not-found exception
    @ExceptionHandler(StudentNotFoundException.class)
    public ModelAndView handleNotFound(StudentNotFoundException ex) {
        ModelAndView mv = new ModelAndView("error/not-found");
        mv.addObject("errorTitle", "Student Not Found");
        mv.addObject("errorMessage", ex.getMessage());
        return mv;
    }

    // Handle access denied
    @ExceptionHandler(AccessDeniedException.class)
    @ResponseStatus(HttpStatus.FORBIDDEN)
    public String handleForbidden(Model model) {
        model.addAttribute("errorMessage", "You do not have permission.");
        return "error/forbidden";
    }

    // Catch-all handler
    @ExceptionHandler(Exception.class)
    public ModelAndView handleAll(Exception ex) {
        ModelAndView mv = new ModelAndView("error/generic");
        mv.addObject("errorMessage", "An unexpected error occurred: " + ex.getMessage());
        return mv;
    }
}
```

Topic 7: Spring Data JPA

7.1 What is JPA and Spring Data JPA?

JPA (Java Persistence API) is a Java specification for mapping Java objects to relational database tables (ORM — Object Relational Mapping). **Hibernate** is the most popular implementation of JPA.

Spring Data JPA is a layer on top of JPA/Hibernate that eliminates even more boilerplate. Instead of writing EntityManager queries manually, you just define an **interface extending JpaRepository**, and Spring generates the implementation automatically.

Layer	Technology	What you write
Plain JDBC	java.sql.*	Connection, Statement, ResultSet, close() — all manual
Spring JDBC	JdbcTemplate	SQL queries + RowMapper
JPA / Hibernate	EntityManager	JPQL queries, @Entity classes
Spring Data JPA	JpaRepository	Just an interface! Spring generates the implementation

7.2 Creating a Spring Data JPA Project

[illegible]

7.3 Entity Class and Entity Mapping

An **@Entity** class maps to a database table. Each field maps to a column. JPA relationships (**@OneToOne**, **@OneToMany**, **@ManyToOne**, **@ManyToMany**) map foreign keys and join tables.

[illegible]

7.4 Repository: CrudRepository and JpaRepository

Spring Data JPA repositories are interfaces. You don't write implementations — Spring generates them at runtime.

Interface	Extends	Key Methods Provided
Repository<T,ID>	—	Marker only
CrudRepository<T,ID>	Repository	save, findById, findAll, delete, count, existsById
PagingAndSortingRepository	CrudRepository	findAll(Sort), findAll(Pageable)
JpaRepository<T,ID>	PagingAndSorting	All above + flush, saveAll, deleteAllInBatch, getOne

[illegible]

7.5 JPQL (Java Persistence Query Language)

JSQL is a query language similar to SQL but operates on **entity objects and their fields**, not on database tables and columns. It is database-independent (works with MySQL, PostgreSQL, Oracle, etc.).


```

@Repository
public class ProductCriteriaDao {

    @PersistenceContext
    private EntityManager em;

    // Dynamic search – apply only the filters that have values
    public List<Product> dynamicSearch(String name, BigDecimal minPrice,
                                      BigDecimal maxPrice, Category category) {
        CriteriaBuilder cb = em.getCriteriaBuilder();
        CriteriaQuery<Product> cq = cb.createQuery(Product.class);
        Root<Product> root = cq.from(Product.class);

        List<Predicate> predicates = new ArrayList<>();

        if (name != null && !name.isEmpty()) {
            predicates.add(cb.like(cb.lower(root.get("name")),
                                   "%" + name.toLowerCase() + "%"));
        }
        if (minPrice != null) {
            predicates.add(cb.greaterThanOrEqualTo(root.get("price"), minPrice));
        }
        if (maxPrice != null) {
            predicates.add(cb.lessThanOrEqualTo(root.get("price"), maxPrice));
        }
        if (category != null) {
            predicates.add(cb.equal(root.get("category"), category));
        }

        cq.select(root)
           .where(predicates.toArray(new Predicate[0]))
           .orderBy(cb.asc(root.get("price")));

        return em.createQuery(cq).getResultList();
    }

    // Criteria with JOIN
    public List<Employee> findEmployeesInDept(String deptName) {
        CriteriaBuilder cb = em.getCriteriaBuilder();
        CriteriaQuery<Employee> cq = cb.createQuery(Employee.class);
        Root<Employee> root = cq.from(Employee.class);
        Join<Employee, Department> deptJoin = root.join("department");

        cq.select(root).where(cb.equal(deptJoin.get("name"), deptName));
        return em.createQuery(cq).getResultList();
    }
}

```

7.7 Spring Data JPA and Transactions

Spring Data JPA repositories are **transactional by default**. All write methods (save, delete) are annotated with `@Transactional` internally. Read methods run without a transaction by default. In your service layer, you control transactions explicitly with `@Transactional`.


```

@Service
public class OrderService {

    @Autowired private OrderRepository orderRepo;
    @Autowired private ProductRepository productRepo;
    @Autowired private CustomerRepository customerRepo;

    // Both order save AND product stock update must succeed or both rollback
    @Transactional
    public Order placeOrder(int customerId, int productId, int qty) {
        Customer customer = customerRepo.findById(customerId)
            .orElseThrow(() -> new RuntimeException("Customer not found"));

        Product product = productRepo.findById(productId)
            .orElseThrow(() -> new RuntimeException("Product not found"));

        if (product.getStock() < qty)
            throw new InsufficientStockException("Not enough stock for: " + product.getName());

        // Reduce stock
        product.setStock(product.getStock() - qty);
        productRepo.save(product);

        // Create order
        Order order = new Order(customer, product, qty,
            product.getPrice().multiply(BigDecimal.valueOf(qty)));
        return orderRepo.save(order);

        // If any step above fails → entire transaction rolls back
        // Neither order nor stock change is committed
    }

    @Transactional(readOnly = true)
    public List<Order> getOrdersByCustomer(int customerId) {
        return orderRepo.findByCustomerId(customerId);
    }
}

```

7.8 Testing Spring Data JPA

@DataJpaTest is a Spring Boot test slice annotation that loads only the JPA-related configuration (entities, repositories) and uses an **in-memory H2 database** by default. Each test runs in a transaction that is automatically **rolled back** after the test, so tests do not leave data behind.

```

@DataJpaTest
class ProductRepositoryTest {

    @Autowired
    private ProductRepository productRepository;

    // TestEntityManager – helper for setting up test data
    @Autowired
    private TestEntityManager entityManager;

    @BeforeEach
    void setUp() {
        Product p1 = new Product("Laptop", new BigDecimal("75000"), 10, Category.ELECTRONICS);
        Product p2 = new Product("T-Shirt", new BigDecimal("500"), 50, Category.CLOTHING);
        entityManager.persistAndFlush(p1);
        entityManager.persistAndFlush(p2);
    }

    @Test
    void testFindByCategory() {
        List<Product> electronics = productRepository.findByCategory(Category.ELECTRONICS);
        assertEquals(1, electronics.size());
        assertEquals("Laptop", electronics.get(0).getName());
    }

    @Test
    void testSaveAndFindById() {
        Product newProduct = new Product("Headphones", new BigDecimal("3000"), 20, Category.ELECTRONICS);
        Product saved = productRepository.save(newProduct);

        assertNotNull(saved.getId());
        Optional<Product> found = productRepository.findById(saved.getId());
        assertTrue(found.isPresent());
        assertEquals("Headphones", found.get().getName());
    }

    @Test
    void testFindByPriceRange() {
        List<Product> result = productRepository.findByPriceRange(
            new BigDecimal("400"), new BigDecimal("1000"));
        assertEquals(1, result.size());
        assertEquals("T-Shirt", result.get(0).getName());
    }

    @Test
    void testCountByCategory() {
        long count = productRepository.countByCategory(Category.ELECTRONICS);
        assertEquals(1, count);
    }

    @Test
    void testDelete() {
        long beforeCount = productRepository.count();
        productRepository.deleteAll();
        assertEquals(0, productRepository.count());
    }
}

```

Topic 8: Spring REST

8.1 What is REST?

REST (Representational State Transfer) is an architectural style for designing web APIs. A RESTful API exposes **resources** (like Products, Employees, Orders) through **URLs** and uses standard **HTTP methods** to perform operations on them. Data is typically exchanged as **JSON**.

REST Constraints (Principles):

- **Stateless:** Each request must contain all information needed to process it. The server stores no client session state.
- **Client-Server:** Client and server are separate. Client handles UI, server handles data.
- **Uniform Interface:** Resources are identified by URIs. HTTP methods have standard meanings.
- **Cacheable:** Responses can be cached to improve performance.
- **Layered System:** Client doesn't know if it's talking to the actual server or an intermediary (load balancer, cache).

HTTP Method	CRUD Operation	URL Example	Success Response
GET	Read	/api/products or /api/products/1	200 OK
POST	Create	/api/products	201 Created
PUT	Update (full)	/api/products/1	200 OK
PATCH	Update (partial)	/api/products/1	200 OK
DELETE	Delete	/api/products/1	204 No Content

8.2 REST Basics and Annotations

@RestController = **@Controller** + **@ResponseBody**. Every method automatically serialises the return value to JSON (using Jackson) and writes it to the HTTP response. There is no view resolution — it's just data.

[illegible]

```

    }

    // PUT /api/products/5 – full update
    @PutMapping("/{id}")
    public ResponseEntity<Product> update(@PathVariable int id,
                                         @Valid @RequestBody Product product) {
        if (!service.existsById(id)) return ResponseEntity.notFound().build();
        product.setId(id);
        return ResponseEntity.ok(service.save(product));
    }

    // PATCH /api/products/5/stock – partial update (just stock)
    @PatchMapping("/{id}/stock")
    public ResponseEntity<Product> updateStock(@PathVariable int id,
                                              @RequestParam int qty) {
        service.updateStock(id, qty);
        return ResponseEntity.ok(service.findById(id).orElseThrow());
    }

    // DELETE /api/products/5
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable int id) {
        if (!service.existsById(id)) return ResponseEntity.notFound().build();
        service.delete(id);
        return ResponseEntity.noContent().build();           // 204 No Content
    }
}

```

8.3 Handling Request and Response

Spring REST provides annotations to extract data from different parts of an HTTP request, and **ResponseEntity<T>** gives full control over the HTTP response (status code, headers, body).

Annotation	Source	Example
@PathVariable	URL path segment	/products/{id} → int id
@RequestParam	URL query string	/search?name=x → String name
@RequestBody	Request body (JSON→Object)	@RequestBody Product product
@RequestHeader	HTTP header	@RequestHeader("Authorization") String token
@CookieValue	Cookie	@CookieValue("session") String session
ResponseEntity<T>	Full response control	status, headers, body together

[illegible]

8.4 Global Error Handling in Spring REST

@RestControllerAdvice is the REST equivalent of **@ControllerAdvice**. It catches exceptions from all REST controllers globally and returns structured JSON error responses instead of HTML error pages. This ensures your API always returns consistent, parseable error information.

[illegible]

```
// 400 – validation failed (@Valid on @RequestBody)
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, Object>> handleValidation(
    MethodArgumentNotValidException ex, HttpServletRequest req) {
    Map<String, Object> body = new LinkedHashMap<>();
    body.put("status", 400);
    body.put("error", "Validation Failed");
    body.put("timestamp", LocalDateTime.now());
    body.put("path", req.getRequestURI());

    Map<String, String> fieldErrors = new HashMap<>();
    ex.getBindingResult().getFieldErrors()
        .forEach(fe -> fieldErrors.put(fe.getField(), fe.getDefaultMessage()));
    body.put("errors", fieldErrors);

    return ResponseEntity.badRequest().body(body);
}

// 500 – unexpected errors
@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleAll(Exception ex, HttpServletRequest req) {
    ErrorResponse err = new ErrorResponse(
        500, "Internal Server Error", "An unexpected error occurred", req.getRequestURI());
    return ResponseEntity.status(500).body(err);
}

// ■■ Example JSON response for 404: ████████████████████████████████████
// {
//   "status": 404,
//   "error": "Not Found",
//   "message": "Product not found with ID: 99",
//   "path": "/api/products/99",
//   "timestamp": "2026-05-14T12:00:00"
// }
```

8.5 Spring REST HTTP Methods Deep Dive

Understanding when and how to use each HTTP method correctly makes your API truly RESTful and intuitive to consume.

[illegible]

[illegible]

```
void testGetAllIntegration() {
    ResponseEntity<List> response =
        restTemplate.getForEntity("/api/products", List.class);
    assertEquals(HttpStatus.OK, response.getStatusCode());
    assertNotNull(response.getBody());
}

@Test
void testCreateAndGet() {
    Product p = new Product("Camera", new BigDecimal("25000"), 5, Category.ELECTRONICS);
    ResponseEntity<Product> created =
        restTemplate.postForEntity("/api/products", p, Product.class);

    assertEquals(HttpStatus.CREATED, created.getStatusCode());
    assertNotNull(created.getBody().getId());

    // Now fetch it
    ResponseEntity<Product> fetched = restTemplate.getForEntity(
        "/api/products/" + created.getBody().getId(), Product.class);
    assertEquals("Camera", fetched.getBody().getName());
}
}
```

r Your Cognizant Interim! ■

Key areas to focus on:

- ✓ @Transactional — propagation, isolation, rollbackFor
- ✓ @Valid + BindingResult + @ControllerAdvice
- ✓ JpaRepository derived queries and @Query
- ✓ REST HTTP methods, status codes, ResponseEntity
- ✓ AOP — @Before, @Around, pointcut expressions
- ✓ Bean scopes — singleton vs prototype
- ✓ Constructor injection vs setter injection